



Universidad de Sonora | Teoría de la Computación

Lenguajes Recursivos y Problemas Decidibles

Sección 8.2

Integrantes: Gael Juan Valenzuela · Angel Fernando Borquez Guerrero · Owen Adiel Solis Zatarain
· Jorge Alan Ahumada Herrera

2 de mayo de 2026

Máquinas de Turing como Algoritmos

- No toda mt (Maquina de Turing) modela correctamente un algoritmo.
- **Ejemplo:** la mt M que imprime indefinidamente

0 1 0 1 0 1 $\dots$

nunca alcanza una configuración de paro.

- No produce una respuesta definida
 $\Rightarrow$ **no es un algoritmo.**
- Un buen modelo de algoritmo: mt que **siempre termina** y produce una respuesta, desde cualquier configuración inicial.

Entrada y salida de una MT

Sea M una mt que siempre alcanza una configuración de paro. Dada la configuración inicial $(q_0, \underline{0}11101)$:

- $w = 011101$ es la *entrada del algoritmo*.
- El cómputo llega a $(q, 000001\sqcup)$, sin importar si $q \in F$ o no.
- Si $F = \emptyset$, la cinta es la *salida del algoritmo*.

Se escribe $M(w) = 000001$, es decir:

$$(q_0, \underline{0}11101) \vdash^* (q, 000001\sqcup)$$

Descripción de la máquina

La mt M recibe un número binario w y calcula:

$$M(w) = \begin{cases} w - 1 & \text{si } w \text{ es impar,} \\ w + 1 & \text{si } w \text{ es par.} \end{cases}$$

Observaciones:

- w par $\Leftrightarrow$ termina en 0; impar $\Leftrightarrow$ termina en 1.
- Restar 1 al impar: reemplazar el último 1 por 0.
- Sumar 1 al par: reemplazar el último 0 por 1.

Se tiene $F = \emptyset$.

Tabla de transiciones de M :

	0	1	$\sqcup$
$\rightarrow q_0$	$(q_0, 0, \rightarrow)$	$(q_0, 1, \rightarrow)$	$(q_1, \sqcup, \leftarrow)$
q_1	$(q_2, 1, \rightarrow)$	$(q_2, 0, \rightarrow)$	—
q_2	—	—	—

q_0 : Recorre la cinta a la derecha sin modificar nada.

q_1 : Modifica el último símbolo leído.

q_2 : Estado de paro; la salida queda en cinta.

$$17 = 10001_2 \text{ (impar)} \xrightarrow{M} 10000_2 = 16$$

	Estado	Cinta
inicio	q_0	<u>1</u> 0001
	q_0	1 <u>0</u> 001
	q_0	10 <u>0</u> 01
	q_0	100 <u>0</u> 1
	q_0	1000 <u>1</u>
	q_0	10001 <u>␣</u>
	q_1	1000 <u>1</u>
fin	q_2	10000 <u>␣</u>

Análisis paso a paso

1. q_0 avanza hacia la derecha **sin modificar** ningún símbolo.
2. Al leer $\sqcup$: transita a q_1 y **retrocede** una posición.
3. q_1 lee el último dígito (1, impar) y lo **reemplaza por 0**.
4. q_2 : configuración de paro. La salida en cinta es $10000 = 16$.

Notación compacta:

$$(q_0, \underline{1}0001) \vdash^* (q_2, 10000\underline{\sqcup})$$

$$M(10001) = 10000$$

Problemas de Decisión y Lenguajes Recursivos

Una mt M con $F \neq \emptyset$ que **no entra en ciclos infinitos** puede resolver un **problema de decisión**.

Ejemplos de problemas de decisión:

- ¿Es un número par?
- ¿Contiene un texto la palabra **sonora**?
- ¿Tiene un programa los paréntesis equilibrados?
- ¿Es una cadena un palíndromo?

La entrada w es, respectivamente, el número, el texto, el programa o la cadena.

Las dos posibilidades de $M(w)$

Dada una mt M y una entrada w :

- M **no se detiene**:

$$M(w) = \text{loop}$$

- M **para** y devuelve:

$$M(w) = \begin{cases} \text{true} & \text{si } w \in L(M), \\ \text{false} & \text{si } w \notin L(M). \end{cases}$$

La salida depende de si el estado de paro es **de aceptación** o no.

Lenguaje Recursivo

Se dice que L es un *lenguaje recursivo* si existe una mt M tal que $L = L(M)$, y a partir de **cualquier configuración inicial** se llega a una **configuración de paro**.

- Son los lenguajes **decidibles**: existe un algoritmo que siempre responde si $w \in L$ o $w \notin L$.
- La mt decisora **siempre para**; nunca entra en loop.
- Los lenguajes no recursivos corresponden a **problemas indecidibles**.

Pregunta clave: ¿Existen lenguajes que *ninguna* mt puede decidir?

La respuesta es **sí**. Eso estudia la *indecidibilidad*, que se formalizará en los siguientes slides.

El límite matemático: Indecidibilidad

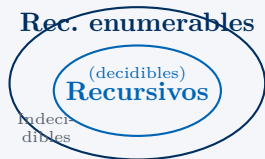
Problema indecidible

Un *problema indecidible* es un problema de decisión caracterizado por un **lenguaje no recursivo**.

Un problema que **no** es indecidible se llama **decidible**.

- No existe ninguna mt que, dada cualquier entrada, siempre responda true o false.
- Algunos lenguajes **no** pueden ser reconocidos por ningún algoritmo.
- La indecidibilidad no es una limitación tecnológica: es un **límite matemático** fundamental.

Jerarquía de lenguajes



Todo lenguaje recursivo es rec. enumerable, pero **no** al revés.

Planteamiento

Se asigna a cada gramática libre de contexto (glc) una **codificación** w sobre un alfabeto fijo Σ .

El lenguaje asociado al problema es:

$$L_G = \{ w \in \Sigma^* : w \text{ es una glc ambigua} \}$$

Resultado: L_G no es recursivo.

Conclusión: El problema de saber si una gramática libre de contexto es ambigua es **indecidable**.

No existe ningún algoritmo que, para toda glc G , determine en tiempo finito si G es ambigua.

¿Qué significa que L_G no sea recursivo?

1. Ninguna mt M puede decidir, para **toda** entrada w , si $w \in L_G$.
2. Podría existir una mt que reconozca algunas gramáticas ambiguas, pero **no todas**: entraría en loop para ciertos casos.
3. El lenguaje L_G pertenece a los **rec. enumerables no recursivos**.

Este es uno de los problemas indecibles más importantes para compiladores y procesamiento de lenguajes.

La Tesis de Church-Turing

La Tesis Central

«Todo algoritmo o procedimiento mecánico de cómputo puede ser llevado a cabo por una **Máquina de Turing**.»

- **Equivalencia histórica:** Alonzo Church (λ -cálculo) y Alan Turing (mt) formularon modelos distintos que resultaron ser **idénticos** en poder.
- **Turing-Compleitud:** Cualquier lenguaje de programación moderno es equivalente a una mt.

¿Qué es computable?

La tesis define los límites de lo que es "calculable". Si una mt no puede resolver un problema, se considera que dicho problema es **intrínsecamente no computable**.

- **¿Por qué es una Tesis?** No es un teorema matemático porque el concepto de "algoritmo" es una noción intuitiva, no una definición formal previa.
- **Evidencia empírica:** En casi un siglo, no se ha hallado ningún modelo físico o lógico que supere el poder de cómputo de la Máquina de Turing.

El límite del conocimiento:

Esta tesis marca la frontera final de la informática: establece que hay problemas que la lógica humana puede entender, pero que **ninguna máquina** podrá resolver jamás.

Implicación en Decidibilidad

Un problema es **decidible** si y solo si existe una mt que lo resuelva. La tesis asegura que esta definición es universal.

Verificar que un lenguaje sea
recursivo

Planteamiento

Consideremos el lenguaje:

$$L_{012} = \{ 0^n 1^n 2^n : n > 0 \}$$

El ejercicio pide verificar que L_{012} es **recursivo**.

Para demostrarlo, debemos describir una mt que decida este lenguaje.

Objetivo

Queremos construir una mt M que:

1. acepte toda cadena de la forma $0^n 1^n 2^n$;
2. rechace toda cadena que no tenga esa forma;
3. siempre se detenga.

Si tal máquina existe, entonces L_{012} es recursivo.

Idea principal

Una cadena pertenece a L_{012} si tiene la forma:

$$0^n 1^n 2^n$$

Esto significa que por cada 0 debe existir exactamente un 1 y exactamente un 2.

La mt puede verificar esto marcando símbolos por grupos.

Marcas utilizadas

La máquina reemplaza los símbolos ya procesados por marcas:

$$0 \rightarrow X, \quad 1 \rightarrow Y, \quad 2 \rightarrow Z$$

Donde:

- X representa un 0 ya usado;
- Y representa un 1 ya usado;
- Z representa un 2 ya usado.

Primera revisión

Dada una cadena w , la máquina primero verifica que tenga la forma general:

$$0^+1^+2^+$$

Es decir:

- primero deben aparecer únicamente ceros;
- después únicamente unos;
- al final únicamente doses.

Si aparece un símbolo fuera de orden, la máquina rechaza.

Ejemplos de orden incorrecto:

010122, 001212, 120

Estas cadenas se rechazan porque no respetan el orden $0^+1^+2^+$.

Proceso repetitivo

Después de verificar el orden, la máquina regresa al inicio de la cinta y repite el siguiente proceso:

1. busca el primer 0 sin marcar;
2. si lo encuentra, lo cambia por X ;
3. busca hacia la derecha el primer 1 sin marcar;
4. si lo encuentra, lo cambia por Y ;
5. busca hacia la derecha el primer 2 sin marcar;
6. si lo encuentra, lo cambia por Z .

Interpretación

Cada vuelta del proceso marca un grupo completo:

0, 1, 2

Es decir, la máquina empareja un 0 con un 1 y con un 2.

Si en algún momento encuentra un 0, pero ya no hay un 1 o un 2 disponible, entonces la cadena no tiene la misma cantidad de símbolos y se rechaza.

Cuando ya no hay ceros

El proceso se repite hasta que ya no exista ningún 0 sin marcar.

En ese momento, la máquina realiza una última revisión de la cinta.

Si todos los símbolos fueron marcados, entonces la cinta solo contiene símbolos de la forma:

$$X^n Y^n Z^n$$

Criterio final:

La máquina acepta si solo quedan símbolos marcados:

$$X, \quad Y, \quad Z$$

La máquina rechaza si queda algún símbolo sin marcar:

$$0, \quad 1, \quad 2$$

Esto asegura que había la misma cantidad de 0, 1 y 2.

Entrada

Consideremos la cadena:

000111222

La máquina marca un 0, un 1 y un 2 en cada repetición.

Primera repetición:

$000111222 \Rightarrow X00Y11Z22$

Segunda repetición:

$X00Y11Z22 \Rightarrow XX0YY1ZZ2$

Resultado

Tercera repetición:

$XX0YY1ZZ2 \Rightarrow XXXYYYZZZ$

Ya no quedan ceros sin marcar.

Además, tampoco quedan unos ni doses sin marcar.

Por lo tanto, la máquina acepta:

$000111222 \in L_{012}$

Entrada

Consideremos la cadena:

0011222

La máquina puede marcar dos grupos completos:

$0011222 \Rightarrow X0Y1Z22$

$X0Y1Z22 \Rightarrow XXYYZZ2$

Resultado:

Ya no quedan ceros sin marcar.

Sin embargo, todavía queda un 2 sin marcar:

$XXYYZZ2$

Esto significa que no había la misma cantidad de 0, 1 y 2.

Por lo tanto:

$0011222 \notin L_{012}$

Resumen

Lenguaje recursivo MT que *siempre* para y decide $w \in L$ o no.

Problema decidable Caracterizado por un lenguaje recursivo.

Problema indecidible Lenguaje no recursivo; ningún algoritmo lo resuelve.

Tesis Church-Turing Todo algoritmo es simulable por una MT.

Ejemplo indecidible Ambigüedad de gramáticas libres de contexto.